

# Compiler Design – GATE-level notes

## # 1. Overall Translation Process (HLL → Executable)

...

HLL (source)

↓ (preprocessor: #include, #define → macro expansion)

Pure HLL

↓ (compiler)

Assembly language

↓ (assembler)

Relocatable machine code (object code)

↓ (loader / linker)

Executable code

...

- Preprocessor handles file inclusion and macro expansion.
- Compiler produces assembly.
- Assembler produces relocatable object code.
- Loader/Linker produces absolute executable code.

## ### 2. Phases of a Compiler

...

HLL

↓

Lexical Analyser → Stream of tokens

↓

Syntax Analyser → Parse tree

↓

Semantic Analyser → Parse tree (semantically verified)

↓

Intermediate Code Generation → Three-address code (max 3 addresses)

↓

Code Optimisation

↓

Target Code Generator → Assembly / Machine code

...

- **Symbol Table** is used by almost all phases.
- **Error Handler** interacts with Lexical, Syntax and Semantic phases.

**Frontend** = up to Intermediate Code Generation (Lexical + Syntax + Semantic + ICG)

**Backend** = Code Optimisation + Target Code Generation

### ### 3. Example Walk-through ( $x = a + b * c;$ )

**\*\*Lexical Analysis\*\***

Tokens: `id = id + id \* id`

**\*\*Syntax Analysis\*\*** (CFG used)

...

$S \rightarrow id = E ;$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

...

Parse tree is built.

**\*\*Semantic Analysis\*\***

Type checking + semantic verification of the parse tree.

**\*\*Intermediate Code Generation (Three-address code)\*\***

...

$t1 = b * c$

$t2 = a + t1$

$x = t2$

...

**\*\*Code Optimisation\*\*** (example)

...

$t1 = b * c$

$t2 = a + t1$

...

**\*\*Target Code Generation\*\*** (example assembly)

...

MOV R1, R2

ADD R0, R2

MOV R2, x

...

(with register allocation: `a → R0`, `b → R1`, `c → R2`)

### ### 4. Symbol Table

Each entry stores attributes that help different phases:

1. Name
2. Size
3. Dimension
4. Type
5. Line of declaration
6. Line of usage
7. Address

**\*\*Example\*\***

| Name | Type | Size | Dim | Line of decl | Line of usage | Address |
|------|------|------|-----|--------------|---------------|---------|
| RAVI | char | 4    | 1   | ...          | ...           | ...     |
| AGE  | int  | 2    | 0   | ...          | ...           | ...     |

**\*\*Why dynamic size?\*\***

- Fixed small size → cannot store many variables
- Fixed large size → space wastage
- Dynamic allocation is used.

**\*\*Operations on Symbol Table\*\***

- **\*\*Non-block structured languages\*\*** (single scope)
  - Insert
  - Lookup
- **\*\*Block structured languages\*\*** (nested scopes, redeclaration allowed)
  - Insert
  - Lookup
  - Set
  - Reset

### ### 5. Lexical Analyser

- Converts lexemes → tokens
- Removes white space & comments

Example snippet produces **\*\*25 tokens\*\***.

### ### 6. Syntax Analyser & Grammars

**\*\*Grammar\*\***

$G = (V, T, P, S)$

**\*\*Main purpose of grammar\*\*** → generate strings of the language.

**\*\*Leftmost Derivation (LMD)\*\*** and **\*\*Rightmost Derivation (RMD)\*\***

Example for  $\text{'id + id * id'}$ :

LMD:

...

$E \rightarrow E + E \rightarrow \text{id} + E \rightarrow \text{id} + E * E \rightarrow \text{id} + \text{id} * E \rightarrow \text{id} + \text{id} * \text{id}$

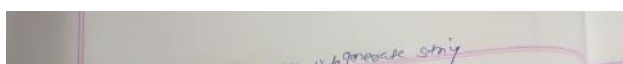
...

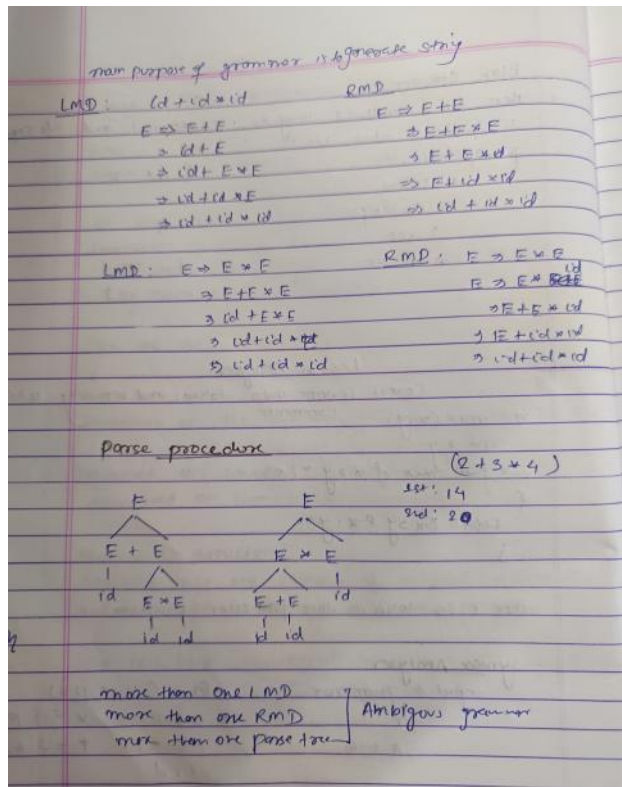
RMD:

...

$E \rightarrow E + E \rightarrow E + E * E \rightarrow E + E * \text{id} \rightarrow E + \text{id} * \text{id} \rightarrow \text{id} + \text{id} * \text{id}$

...





**\*\*Parse Tree\*\*** for expression `(2 + 3 \* 4)` has two different trees → different evaluation order (14 vs 20).

### **\*\*Ambiguous Grammar\*\***

A grammar is ambiguous if a string has:

- more than one LMD, **\*\*or\*\***
- more than one RMD, **\*\*or\*\***
- more than one parse tree.

### **\*\*Examples of ambiguous grammars\*\*** (from notes)

1.  $S \rightarrow aS \mid Sa \mid a$

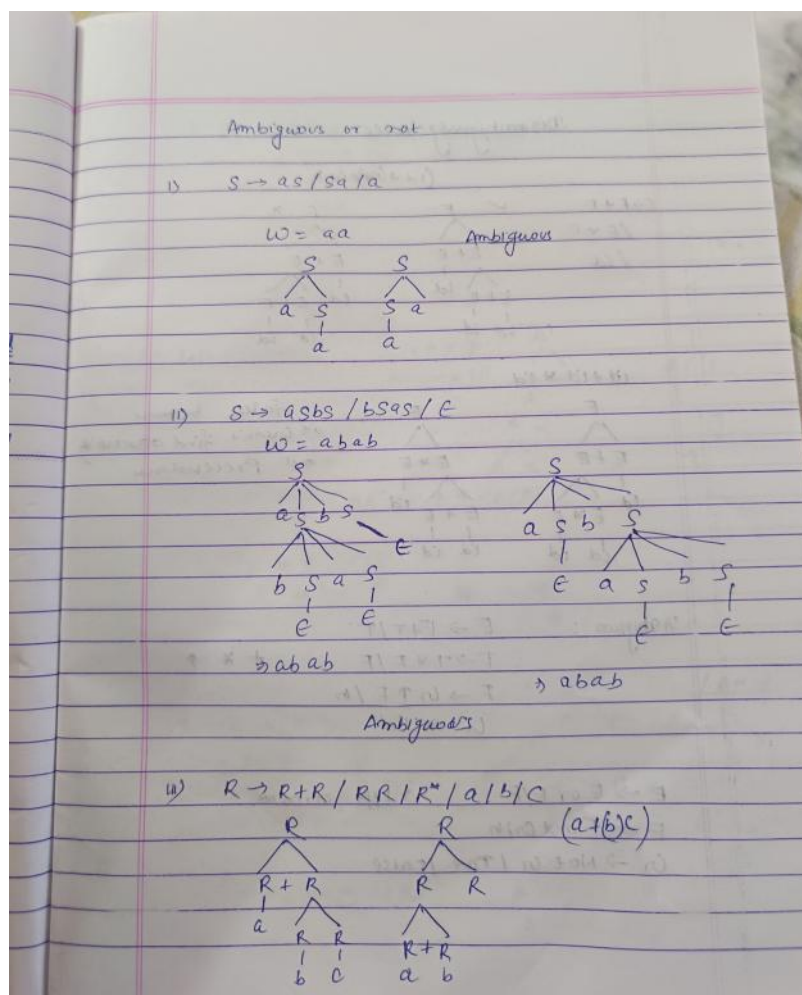
String `aa` has two different parse trees → **\*\*Ambiguous\*\***.

2.  $S \rightarrow aSbS \mid bSaS \mid \epsilon$

String `abab` has two different parse trees → **\*\*Ambiguous\*\***.

3.  $R \rightarrow R + R \mid RR \mid R^* \mid a \mid b \mid c$

Expression `(a + b)c` has multiple parse trees → **\*\*Ambiguous\*\***.



### **\*\*Key takeaways\*\***

- Know exact sequence of phases and what each produces.
- Symbol table attributes & operations (block vs non-block).
- Difference between LMD, RMD and parse tree.
- How to prove a grammar is ambiguous (multiple parse trees / derivations for the same string).
- Three-address code generation for simple expressions.
- Frontend vs Backend division.